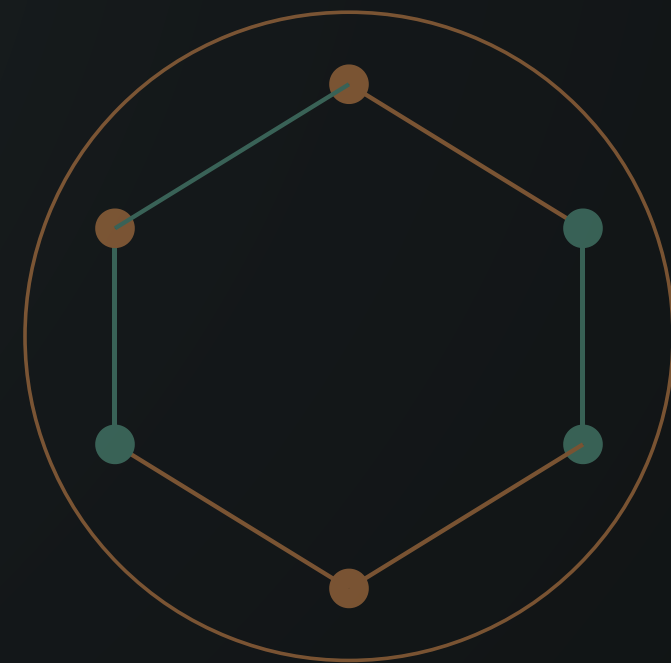


— KIẾN THỨC KỸ THUẬT THỰC TẾ

Vì sao hệ thống sập, và cách làm cho nó không sập

CHỊU TẢI • SỰ CỐ • SAO LƯU • LIVESTREAM TRIỆU NGƯỜI



Sáu câu hỏi buổi này trả lời

- ? Làm sao để hệ thống chịu được 100.000 người mà không sập
- ? Server sập lúc nửa đêm thì xử lý thế nào
- ? Bị tấn công xoá mất dữ liệu thì lấy lại bằng cách nào
- ? TikTok livestream 2-3 triệu người xem, quà bay liên tục, sao vẫn mượt
- ? Cùng một cấu hình server, sao người này chịu 1.000, người kia 10.000
- ? Cùng một trang web, sao máy người này mở nhanh mà máy người kia ì ạch

Sáu câu hỏi này là sáu tầng của cùng một chuỗi

6. Vì sao livestream lại là bài toán khác

3 triệu người

5. Giữ lại dữ liệu khi bị tấn công

Sao lưu

4. Chuẩn bị cho lúc có thứ hỏng

Server sập

3. Một máy không đủ thì thêm máy

100.000 người

2. Đo trước khi đoán

Biết trần của mình ở đâu

1. Một lần bấm tốn bao nhiêu sức của máy

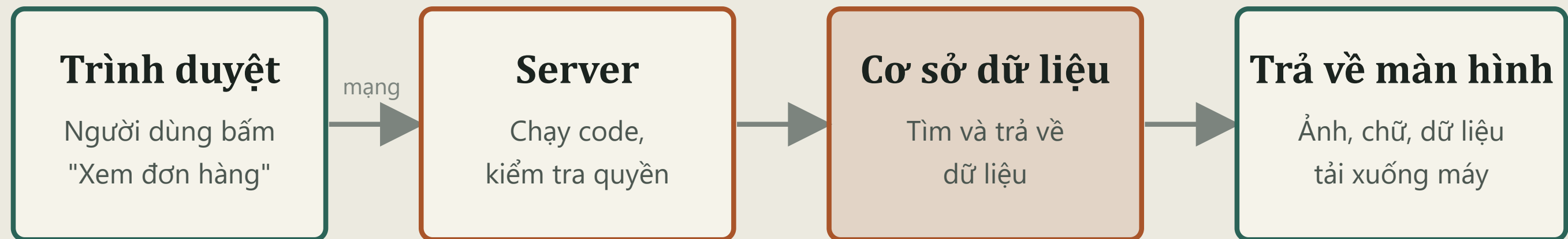
Nền của tất cả

Cùng một server, sao người chịu 1.000 người chịu 10.000

Câu trả lời gần như luôn giống nhau: không phải server yếu, mà là mỗi lần người dùng bấm một nút, hệ thống của bạn bắt máy làm quá nhiều việc thừa.

Cách nghĩ đúng: server giống một nhân viên. Một nhân viên phục vụ được bao nhiêu khách phụ thuộc vào mỗi khách chiếm của họ bao nhiêu phút, chứ không phải nhân viên đó khoẻ cỡ nào.

Một lần bấm nút đi qua những đâu



Mỗi khâu chậm 1 chút, cộng lại thành trang tải 5 giây

Muốn nhanh thì phải biết khâu nào đang ăn thời gian, không đoán mò

Hỏi cơ sở dữ liệu 101 lần thay vì 1 lần

Giống như đi chợ mua 100 món: người khôn đi một chuyến mua hết, người vụng đi về 100 chuyến, mỗi chuyến một món.

CÁCH HAY GẶP TRONG DỰ ÁN SINH VIÊN

Lấy danh sách 100 đơn hàng, rồi lặp từng đơn để lấy tên khách

101 lần hỏi

1 lần lấy danh sách đơn, cộng thêm 100 lần hỏi tên khách. Mỗi lần hỏi mất 5 mili giây thì riêng khâu này đã tốn hơn nửa giây, chỉ để hiện một trang.

CÁCH ĐÚNG

Lấy đơn hàng và tên khách trong cùng một câu hỏi

1 lần hỏi

Dùng phép nối bảng hoặc nạp sẵn dữ liệu liên quan. Cùng kết quả hiển thị, nhưng máy chỉ phải làm việc một lần thay vì 101 lần.

Dấu hiệu nhận biết: trang có ít dữ liệu thì chạy nhanh, thêm dữ liệu vào là chậm dần đều. Đó gần như chắc chắn là lỗi này.

Tìm dữ liệu bằng cách lật từng trang

KHÔNG CÓ CHỈ MỤC

Lật từng trang số để tìm một cái tên

Đọc dòng 1 • dòng 2 • dòng 3 • dòng 4 • ... • dòng 999.998 • dòng 999.999 • dòng 1.000.000

Quét cả 1 triệu dòng • khoảng 2 giây • và càng nhiều dữ liệu càng chậm thêm

CÓ CHỈ MỤC

Mở mục lục, nhảy thẳng tới đúng trang

Tra mục lục



Lấy đúng dòng

Khoảng 20 mili giây • nhanh hơn 100 lần

Việc cần làm: cột nào hay dùng để tìm kiếm hoặc lọc thì tạo chỉ mục cho cột đó. Đây là một dòng lệnh, làm mất 10 giây, nhưng có thể nhanh hơn cả tháng tối ưu code.

Bắt người dùng tải về thứ họ không cần

Đây chính là lý do cùng một trang web, máy người này mở nhanh mà máy người kia ì ạch: mạng và máy mỗi người mỗi khác, nhưng dung lượng bạn bắt họ tải thì bạn quyết định.

THỨ GỬI ĐI	CÁCH LÀM SAI	CÁCH LÀM ĐÚNG
Ảnh sản phẩm	Ảnh gốc 5 MB từ máy ảnh, hiện trong ô 200 pixel	Nén và cắt đúng kích thước hiển thị, còn khoảng 50 KB
Danh sách dữ liệu	Trả về cả 10.000 dòng rồi để trình duyệt tự lọc	Phân trang, mỗi lần 20 dòng
Thư viện giao diện	Nạp cả bộ vài MB để dùng một hiệu ứng nhỏ	Chỉ nạp phần thật sự dùng tới
File tĩnh	Mỗi lần vào trang tải lại từ đầu	Cho phép trình duyệt lưu lại, lần sau không tải nữa

Vì sao cùng server mà chênh nhau 10 lần

Mỗi lần bấm tốn 500 mili giây của server



1.000 người

Cùng server đó, mỗi lần bấm chỉ tốn 50 mili giây



10.000

Không đổi phần cứng. Chỉ sửa cách code hỏi dữ liệu và cách gửi ảnh.

Đây là lý do phải tối ưu trước khi nghĩ tới thuê thêm server. Thêm máy tốn tiền hàng tháng, còn sửa một câu truy vấn và thêm chỉ mục thì miễn phí.

"Chịu tải 100.000 người" nghĩa là gì

Ba con số dưới đây đều được gọi là "100.000 người", nhưng độ khó chênh nhau hàng trăm lần. Nói rõ mình đang nói con số nào là bước đầu tiên.

100.000

Người đã đăng ký tài khoản

Dễ nhất. Họ không vào cùng lúc. Một server nhỏ vẫn kham được

100.000

Người đang online cùng lúc

Khó hơn nhiều. Cần nhiều máy, cần cache, cần hàng đợi

100.000

Lượt gọi mỗi giây

Rất khó. Đây là mức của các nền tảng lớn, cần kiến trúc riêng

Với sản phẩm dự thi của bạn: con số thực tế thường là vài chục tới vài trăm người online cùng lúc. Thiết kế cho đúng mức đó, rồi nói rõ hệ thống mở rộng được tới đâu khi cần.

Đừng nhìn tốc độ trung bình

CON SỐ DỄ ĐÁNH LỪA

Trung bình 200 mili giây

Nghe rất ổn. Nhưng con số này có thể là kết quả của 95 người được phục vụ trong 50 mili giây, còn 5 người phải chờ 3 giây.

Năm người đó chính là những người sẽ bỏ đi và kể với người khác rằng web của bạn chậm.

CON SỐ NÊN NHÌN

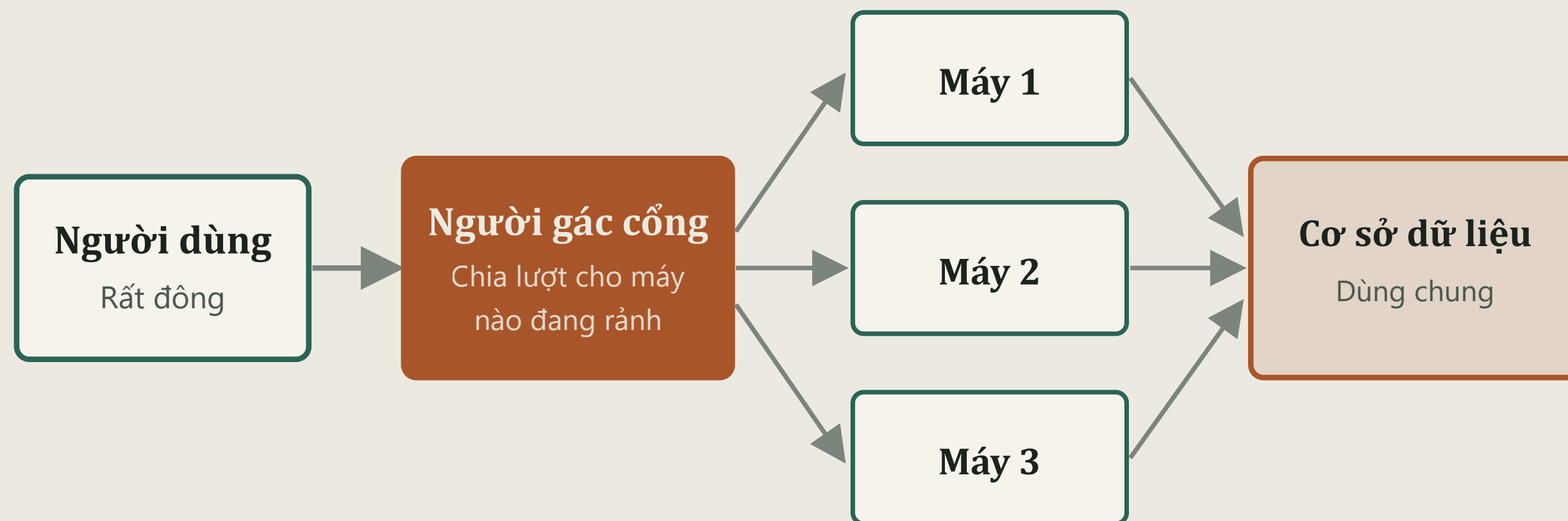
Chậm nhất trong 95% lượt truy cập

Nghĩa là: bỏ qua 5% trường hợp tệ nhất, thì người dùng chậm nhất phải chờ bao lâu.

Nếu con số này dưới 1 giây thì gần như mọi người đều thấy nhanh. Đây mới là thứ phản ánh trải nghiệm thật.

Cách làm đơn giản nhất: ghi lại thời gian xử lý của từng lượt truy cập vào log. Cuối ngày xem lượt nào chậm nhất và tìm hiểu vì sao. Không cần công cụ đắt tiền.

Thêm máy và người gác cổng chia việc



Điều kiện bắt buộc: cả 3 máy phải giống hệt nhau, không máy nào giữ riêng dữ liệu của người dùng

Ghi sẵn câu trả lời hay bị hỏi

Quán ăn đông khách không nấu lại từ đầu mỗi lần có người gọi món phổ biến. Họ nấu sẵn một nồi. Hệ thống cũng vậy.

KHÔNG CÓ BỘ NHỚ ĐỆM

10.000 người xem trang chủ, hỏi cơ sở dữ liệu 10.000 lần

Trong khi nội dung trang chủ giống hệt nhau cho cả 10.000 người đó. Cơ sở dữ liệu làm đi làm lại đúng một việc.

CÓ BỘ NHỚ ĐỆM

Hỏi 1 lần, giữ kết quả trong 60 giây, phục vụ 9.999 người còn lại

Cơ sở dữ liệu chỉ phải làm việc 1 lần. Người dùng cũng nhận kết quả nhanh hơn vì lấy từ bộ nhớ thay vì tra cứu lại.

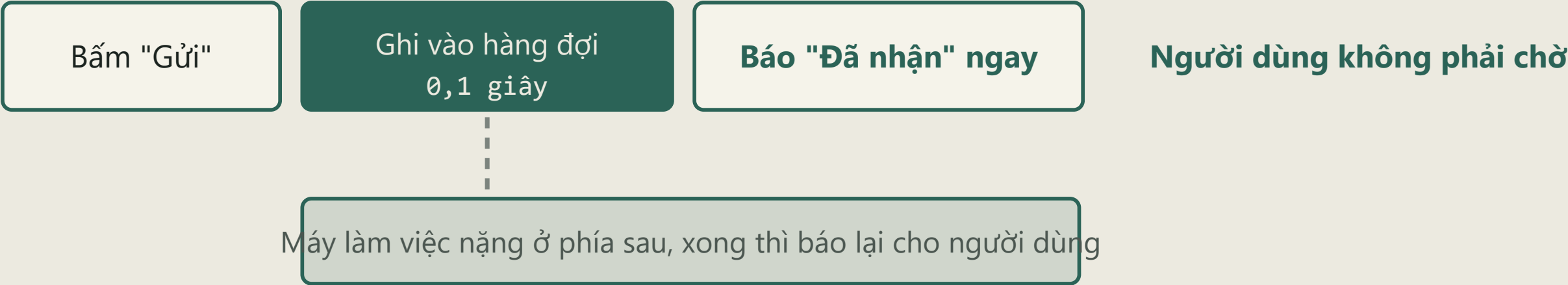
Cái bẫy phải biết: dữ liệu trong bộ nhớ đệm là dữ liệu cũ. Giá sản phẩm hay số dư tài khoản mà lưu đệm 60 giây thì có thể hiện sai. Chỉ lưu đệm thứ chậm thay đổi, và luôn tự hỏi "sai lệch vài giây ở đây có gây hậu quả gì không".

Việc nặng thì đẩy ra sau, đừng bắt người dùng đứng chờ

BẮT NGƯỜI DỪNG CHỜ



TRẢ LỜI NGAY, LÀM SAU



Đây chính là **hàng đợi và cronjob** đã nói ở buổi trước: gửi email, xuất báo cáo, gọi AI xử lý, nén video đều nên đẩy ra sau chứ không làm ngay trong lúc người dùng đang chờ.

Server sẽ sập, câu hỏi chỉ là khi nào

Ổ cứng hỏng, mất điện, nhà mạng đứt cáp, một dòng code mới gây lỗi. Hệ thống tốt không phải là hệ thống không bao giờ hỏng, mà là hệ thống hỏng xong tự đứng dậy được.

01

Tự kiểm tra sức khỏe

Cứ vài giây lại hỏi hệ thống "còn sống không". Không trả lời thì tự khởi động lại, và báo ngay cho người trực

02

Luôn có máy dự phòng

Đừng để chỉ có đúng một máy. Một máy chết thì người gác cổng ngừng chia việc cho nó, chuyển hết sang máy còn lại

03

Quay lại bản cũ trong 1 phút

Vừa cập nhật xong mà lỗi thì phải quay về bản chạy tốt ngay, không ngồi sửa trong lúc người dùng đang không dùng được

Mất một tính năng còn hơn sập cả trang

THIẾT KẾ DỄ GÂY

Phần gợi ý sản phẩm gọi AI, AI lỗi, cả trang trắng

Một tính năng phụ kéo sập toàn bộ trang. Khách không mua được hàng chỉ vì phần gợi ý không chạy.

Nguyên nhân: code không lường trước việc dịch vụ bên ngoài có thể chết hoặc trả lời chậm.

THIẾT KẾ BỀN

AI lỗi thì ẩn phần gợi ý, phần còn lại vẫn chạy

Đặt giới hạn thời gian chờ cho mọi lời gọi ra bên ngoài. Quá hạn thì bỏ qua, hiện nội dung thay thế.

Khách vẫn xem và mua được hàng. Chỉ mất một tính năng phụ mà đa số không để ý.

Áp dụng ngay cho sản phẩm dự thi: phần gợi ý AI của bạn phải có đường lui. Hôm thi mà mạng chậm hoặc dịch vụ AI quá tải, sản phẩm vẫn phải chạy được để demo.

Nhân bản dữ liệu không phải là sao lưu

Đây là hiểu lầm khiến nhiều hệ thống mất sạch dữ liệu dù "đã có 3 bản".



Không với
tới được

Bản sao lưu để tách rời, không ai xoá được
Nằm ở nơi khác, chỉ ghi thêm chứ không cho sửa hay xoá
Đây mới là thứ cứu bạn

Quy tắc 3-2-1 và điều quan trọng nhất

3

bản dữ liệu

Bản đang dùng cộng thêm ít nhất 2 bản sao lưu

2

nơi lưu khác nhau

Không để cả 3 bản trên cùng một máy hoặc cùng một tài khoản

1

bản tách hẳn ra ngoài

Nơi kẻ tấn công chiếm được hệ thống chính cũng không xóa được

Điều nhiều người quên và trả giá đắt nhất: bản sao lưu chưa từng thử phục hồi thì coi như chưa có. Rất nhiều nơi phát hiện file sao lưu bị hỏng hoặc thiếu đúng vào lúc cần dùng nhất. Hãy thử phục hồi định kỳ, và bấm giờ xem mất bao lâu.

Hai câu hỏi phải trả lời trước khi sự cố xảy ra

CÂU HỎI 1

Chấp nhận mất dữ liệu của bao lâu gần nhất

Sao lưu mỗi 24 giờ nghĩa là khi sự cố xảy ra, bạn mất toàn bộ dữ liệu phát sinh trong ngày hôm đó.

Chấp nhận được với blog cá nhân. Không chấp nhận được với hệ thống bán hàng.

CÂU HỎI 2

Chấp nhận hệ thống ngưng chạy bao lâu

Có bản sao lưu nhưng phục hồi mất 6 tiếng thì hệ thống ngưng 6 tiếng.

Biết trước con số này để chuẩn bị, thay vì tới lúc sự cố mới cuống lên đi tìm cách.

Hai con số này quyết định bạn cần sao lưu mấy tiếng một lần và cần chuẩn bị sẵn những gì. Trả lời được hai câu này là đã hơn phần lớn dự án sinh viên.

Vì sao 3 triệu người xem livestream vẫn mượt

Con số có thật: Twitch từng phục vụ hơn 6 triệu người xem đồng thời, World Cup 2022 đạt đỉnh 18 triệu người xem cùng lúc trên một nền tảng. Bí mật nằm ở chỗ video không đi qua server gốc.



Server gốc chỉ phục vụ vài chục máy v

Còn quà và bình luận bay liên tục thì sao

Phần này không dùng được thủ thuật của video, vì nó phải tới ngay. Cách xử lý là chấp nhận một sự thật: không phải con số nào cũng cần chính xác tuyệt đối.

Gộp lại rồi gửi một lần

Thay vì gửi từng bình luận cho từng người, hệ thống gom các bình luận trong nửa giây rồi đẩy một lượt. Giảm hàng nghìn lần số lần gửi

Không hiện hết mọi thứ

Với 3 triệu người, không ai đọc kịp tất cả bình luận. Hệ thống chỉ hiện một phần, và ưu tiên bình luận có quà hoặc của người nổi bật

Số liệu trễ vài giây cũng được

Số người xem hay tổng quà hiện chậm 2 giây thì không ai phàn nàn. Chấp nhận điều đó giúp hệ thống nhẹ đi rất nhiều

Bài học rút ra cho sản phẩm của bạn: hãy tự hỏi chỗ nào trong hệ thống chấp nhận được sai lệch vài giây. Số đơn hàng trong ngày hiện chậm 5 giây thì không sao, nhưng số tiền thanh toán thì phải chính xác tuyệt đối. Phân biệt được hai loại này là biết chỗ nào được phép làm nhẹ đi.

Đừng xây hệ thống cho 3 triệu người khi bạn có 50 người dùng

SAI LẦM THƯỜNG GẶP

Học được kiến trúc lớn rồi đem áp hết vào sản phẩm nhỏ

Chia nhỏ hệ thống thành chục dịch vụ, thêm mấy tầng trung gian, dựng cả cụm máy cho một ứng dụng có 50 người dùng.

Kết quả: mất thời gian gấp ba, nhiều chỗ hỏng hóc hơn, và tới hôm demo thì không chạy được.

CÁCH LÀM ĐÚNG

Làm đơn giản nhất có thể, nhưng biết đường mở rộng

Một máy, một cơ sở dữ liệu, code sạch, có chỉ mục, có sao lưu. Chạy tốt cho vài trăm người dùng.

Khi giám khảo hỏi "mở rộng thế nào", bạn trả lời được rõ ràng từng bước. Đó mới là điều họ muốn nghe.

Thang điểm cuộc thi chấm **tính thực tiễn 25 điểm** và **mức độ hoàn thiện 20 điểm**. Không có mục nào chấm việc hệ thống chịu được bao nhiêu triệu người.

Bảy việc làm được ngay tuần này

- 1 Mở trang chậm nhất của sản phẩm, **đếm xem nó hỏi cơ sở dữ liệu bao nhiêu lần**. Trên 10 lần cho một trang là có vấn đề
- 2 **Tạo chỉ mục** cho những cột hay dùng để tìm kiếm và lọc
- 3 **Nén lại toàn bộ ảnh** đang dùng trong sản phẩm, và phân trang cho mọi danh sách dài
- 4 **Đặt giới hạn thời gian chờ** cho mọi lời gọi ra dịch vụ bên ngoài, nhất là phần gọi API
- 5 Đẩy việc nặng như gửi email và xuất báo cáo **ra khỏi lúc người dùng đang chờ**
- 6 **Bật sao lưu tự động**, và quan trọng nhất là thử phục hồi một lần cho biết nó có chạy không
- 7 Ghi lại thời gian xử lý vào log để **biết trang nào đang chậm**, thay vì đoán